RE Meet 3

Automata Learning 1: Sequential Data

Sequence data refers to a type of data where the order of elements is significant, and the elements are typically arranged in a specific sequence or chronological order. In sequence data, each element in the sequence is related to the ones that come before and after it, and the way these elements are ordered provides context, meaning, and structure to the data.

Sequential data arises in numerous applications of Machine Learning:

- Natural language processing
- Computational biology
- Time series analysis
- Sequential decision-making
- Robotics

Learning from sequential data requires specialised algorithms since ML assume data to be of vector form of fixed dimension but sequences can have arbitrary length similar to trees, graphs.

Function on Strings

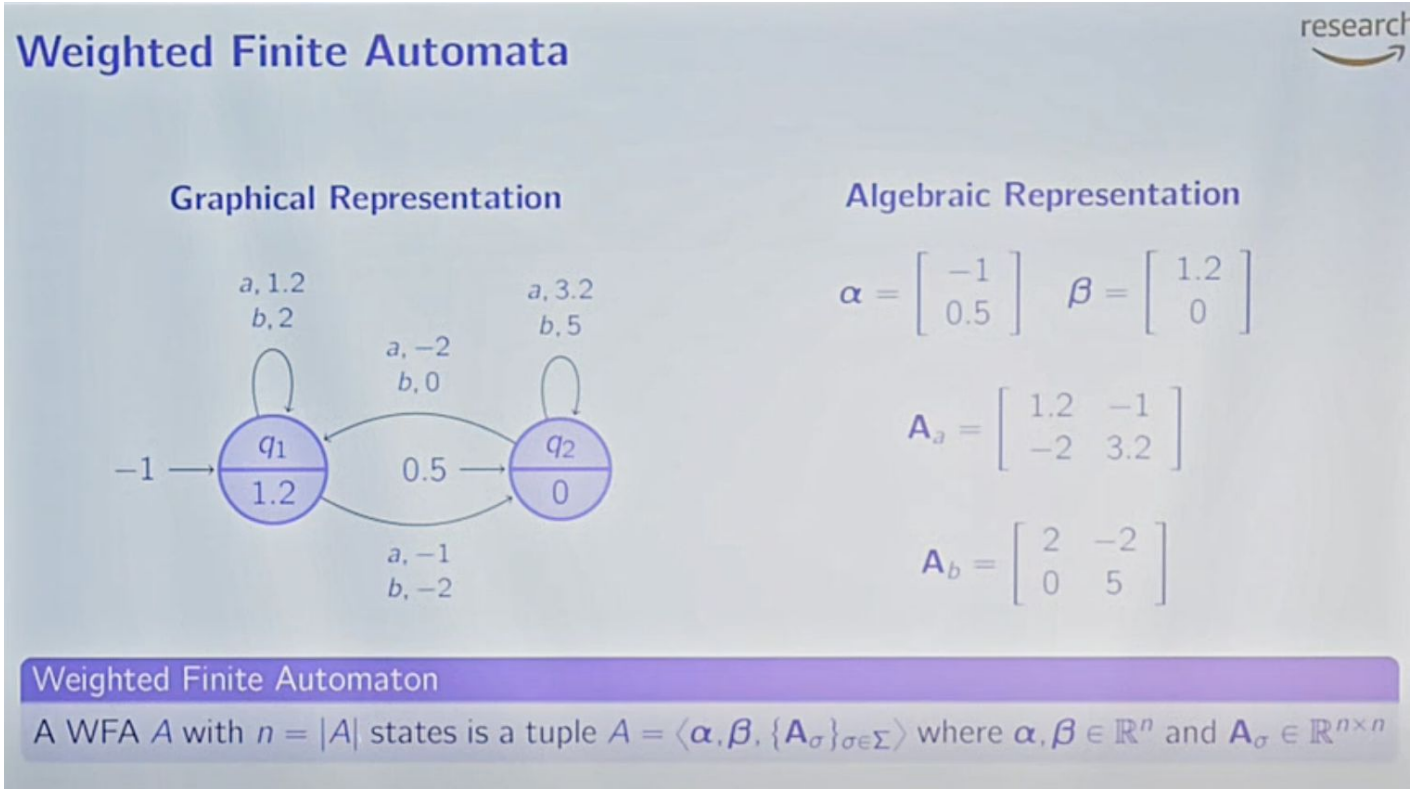
We want to learn a function f : Σ* -> R, strings of finite alphabet to data

Examples: likelihood of observing a sentence in a specific natural language, probability that a sequence of packets will successfully transmit a message

Weighted Language: We can think of the function f as a dictionary that assigns a numerical "weight" or "score" to every possible string we can create from your alphabet (Σ*). For example, in a language model, the string "hello world" gets a high probability (a weight), while "world hello" gets a lower one. This collection of all strings and their assigned weights is called a "weighted language."

Infinite-Dimensional Object: The set of all possible strings (Σ*) is infinite. Since the function f has to assign a value to each of these infinitely many strings, it can't be described by a simple list of values. In mathematical terms, it's considered an "infinite-dimensional object," which is very difficult to work with directly on a computer. In other words: $f \in \mathbb{R}^{\Sigma^*}$

Solution : We use weighted finite automata



Alpha: vector representing the starting weight for each state
Beta: vector representing the final weight for each state
$\mathbf{A_a}$: Matrix storing transition weight for alphabet 'a', a_{ij}-> the transition weight to go from state i to state j for alphabet a.

Language of WFA

$$f_A(x_1 \cdots x_T) = \sum_{q_0, q_1, \ldots, q_T \in [n]} \boldsymbol{\alpha}(q_0) \left(\prod_{t=1}^{T} \mathbf{A}_{x_t}(q_{t-1}, q_t) \right) \boldsymbol{\beta}(q_T)$$

$$= \boldsymbol{\alpha}^\top \mathbf{A}_{x_1} \cdots \mathbf{A}_{x_T} \boldsymbol{\beta} = \boldsymbol{\alpha}^\top \mathbf{A}_x \boldsymbol{\beta}$$

We get the weight from this function

A weighted language is rational if there exists WFA A such that $f=f_A$.

Smallest number of states of a WFA is rank(f). A WFA is minimal if $|A|=\text{rank}(f_A)$

Observation: Minimal A cannot be unique, since we can have any matrix Q such that

$$\boldsymbol{\alpha}^\top \mathbf{A}_{x_1} \cdots \mathbf{A}_{x_T} \boldsymbol{\beta} = (\boldsymbol{\alpha}^\top \mathbf{Q})(\mathbf{Q}^{-1} \mathbf{A}_{x_1} \mathbf{Q}) \cdots (\mathbf{Q}^{-1} \mathbf{A}_{x_T} \mathbf{Q})(\mathbf{Q}^{-1} \boldsymbol{\beta})$$

Examples: DFA, HMM

researc

Deterministic Finite Automata

- Weights in $\{0, 1\}$
- Initial: α indicator for initial state
- Final: β indicates accept/reject state
- Transition: $\mathbf{A}_\sigma(i, j) = \mathbb{I}[i \xrightarrow{\sigma} j]$
- $f_A : \Sigma^* \to \{0, 1\}$ defines regular language

Hidden Markov Model

- Weights in $[0, 1]$
- Initial: α distribution over initial state
- Final: β vector of ones
- Transition: $\mathbf{A}_\sigma(i, j) = \mathbb{P}[i \xrightarrow{\sigma} j] = \mathbb{P}[i \to j]\mathbb{P}[i \xrightarrow{\sigma}]$
- $f_A : \Sigma^* \to [0, 1]$ defines dynamical system

I: Indicator function, whether the input inside the bracket is true or false(exists or not exists)

Hankel Matrices

$$
\mathbf{H}_f = \begin{array}{c} \\ \epsilon \\ a \\ b \\ \vdots \\ p \\ \vdots \end{array}
\begin{array}{c}
\begin{array}{cccccc} \epsilon & a & b & \cdots & s & \cdots \end{array} \\
\begin{bmatrix}
f(\epsilon) & f(a) & f(b) & & \vdots & \\
f(a) & f(aa) & f(ab) & & \vdots & \\
f(b) & f(ba) & f(bb) & & \vdots & \\
 & & & & & \\
\cdots & \cdots & \cdots & & f(p \cdot s) & \\
 & & & & &
\end{bmatrix}
\end{array}
$$

The value in the cell at the intersection of a row labeled p (prefix) and a column labeled s (suffix) is given by the function f(p·s), where p·s is the string formed by concatenating the prefix and the suffix.

Example from the slide:

Row a, Column b: The value is f(ab)

Row b, Column ba: The value is f(bba)

This matrix, H_f, contains all the information about the function f

Flies-Kronecker Theorem:

The rank of H_f is finite if and only if f is rational, or rank(H_f) = rank(f)

We can think it like describing colors. We can have an infinite number of colors (like maroon, crimson, scarlet), but they can all be described as a combination of just three "basis" colors: red, green, and blue. So, the "rank" of the color space is 3.

From theorem to WFA

We have $f(p_1 \cdots p_T s_1 \cdots s_{T'}) = \alpha^\top \mathbf{A}_{p_1} \cdots \mathbf{A}_{p_T} \mathbf{A}_{s_1} \cdots \mathbf{A}_{s_{T'}} \beta$

Now we can introduce a new alphabet a such that the matrix becomes

$$f(p_1 \cdots p_T \sigma s_1 \cdots s_{T'}) = \alpha^\top \mathbf{A}_{p_1} \cdots \mathbf{A}_{p_T} \mathbf{A}_a \mathbf{A}_{s_1} \cdots \mathbf{A}_{s_{T'}} \beta$$

$$\mathbf{H}_\sigma = \begin{array}{cc} & s \\ p & \begin{bmatrix} & \vdots & \\ \cdots & f(pas) & \cdots \\ & \vdots & \end{bmatrix} \end{array} = \begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \bullet & \bullet & \bullet \\ \cdot & \cdot & \cdot \end{bmatrix} \begin{bmatrix} \bullet & \bullet & \bullet \\ \bullet & \bullet & \bullet \\ \bullet & \bullet & \bullet \end{bmatrix} \begin{bmatrix} \cdot & \cdot & \bullet & \cdot & \cdot \\ \cdot & \cdot & \bullet & \cdot & \cdot \\ \cdot & \cdot & \bullet & \cdot & \cdot \end{bmatrix}$$

Algebraically: Factorizing $\mathbf{H}$ lets us solve for $\mathbf{A}_a$

$$\mathbf{H} = \mathbf{P}\ \mathbf{S} \quad \Longrightarrow \quad \mathbf{H}_\sigma = \mathbf{P}\ \mathbf{A}_\sigma\ \mathbf{S} \quad \Longrightarrow \quad \mathbf{A}_\sigma = \mathbf{P}^+\ \mathbf{H}_\sigma\ \mathbf{S}^+$$

Moore Penrose Pseudo-Inverse

For any $\mathbf{M} \in \mathbb{R}^{n \times m}$ there exists a unique *pseudo-inverse* $\mathbf{M}^+ \in \mathbb{R}^{m \times n}$ satisfying:

- $\mathbf{M}\mathbf{M}^+\mathbf{M} = \mathbf{M}$, $\mathbf{M}^+\mathbf{M}\mathbf{M}^+ = \mathbf{M}^+$, and $\mathbf{M}^+\mathbf{M}$ and $\mathbf{M}\mathbf{M}^+$ are symmetric
- If $\text{rank}(\mathbf{M}) = n$ then $\mathbf{M}\mathbf{M}^+ = \mathbf{I}$, and if $\text{rank}(\mathbf{M}) = m$ then $\mathbf{M}^+\mathbf{M} = \mathbf{I}$
- If $\mathbf{M}$ is square and invertible then $\mathbf{M}^+ = \mathbf{M}^{-1}$

Given a system of linear equations $\mathbf{M}\mathbf{u} = \mathbf{v}$, the following is satisfied:

$$\mathbf{M}^+\mathbf{v} = \underset{\mathbf{u} \in \text{argmin} \|\mathbf{M}\mathbf{u}-\mathbf{v}\|_2}{\text{argmin}} \|\mathbf{u}\|_2 .$$

Primary Goal: First, find all vectors u that make the error ||Mu - v|| as small as possible. This term measures the difference between what you get (Mu) and what you want (v). Minimizing this is the classic "least-squares" problem.

Tie-Breaker: If there's more than one u that perfectly minimizes this error (which happens in underdetermined systems), pick the one that has the smallest length or magnitude (||u||). This is called the "minimum norm" solution.

Important points:

If the system is underdetermined(infinite solutions): we have many matrices for which |mu-v|=0

So tiebreaker rule applies here(shortest vector)

If the system is overdetermined(no solution): we just take the u that takes Mu closest to v thus having the minimum error.

WFA Reconstruction from Finite Hankel Sub Blocks

The behavior of the hidden machine is described by an infinitely large table called a **Hankel matrix**. Since we can't use an infinite table, we take a finite snapshot, called a **sub-block (H)**.

This snapshot must be "good enough," meaning it's large enough to capture the machine's full complexity. We know it's good enough if its **rank** (a measure of complexity) is equal to n, the number of states in the hidden machine.

Step 1: Factor the Table

we break the data table H into two smaller, skinnier tables, P and S. This process, called rank factorization. The "inner" size of these new tables will be n, which is exactly the number of states we're looking for.

- $H = P \cdot S$

Step 2: Find the Start and End Rules

The start and end rules for the machine are hiding in plain sight within the new tables. Since we have chosen particular rows and columns, we had to choose the alpha and beta in these.

- **Start Weights (α):** The starting instructions are simply the **first row** of table P.
- **Final Weights (β):** The ending instructions are the **first column** of table S.

Step 3: Find the Transition Rules

To find the rule for a character like 'a' (A_a), you need a slightly different "shifted" data table, H_a.

- You then use a formula that combines your skinny tables (P and S) with this new shifted table (H_a) using a tool called the **pseudo-inverse** (a smart way to divide matrices).
- $A_a = P^+ \cdot H_a \cdot S^+$

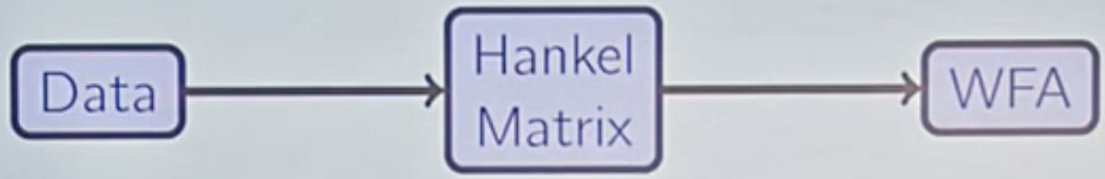
This calculation reveals exactly how the machine transitions from one state to another when it reads the character 'a'. You repeat this for every character to find all the rules.

After these three steps, you have all the pieces (α, β, and all A matrices) and have successfully reconstructed the smallest, most efficient machine that could have produced your data.

Our current roadmap

WFA Learning Algorithms via the Hankel Trick

resea



1. Estimate a Hankel matrix from data
   - For stochastic automata: counting empirical frequencies
   - In general: empirical risk minimization
   - Inductive bias: enforcing low-rank Hankel will yield less states in WFA
   - Parameters: rows and columns of Hankel sub-block
2. Recover a WFA from the Hankel matrix
   - Direct application of WFA reconstruction algorithm

Norms on WFA

Let p,q belong to [1,infinty] be Holder conjugate 1/p+1/q=1

The (p, q)-norm of a WFA A is given by

$$\|A\|_{p,q} = \max\left\{\|\boldsymbol{\alpha}\|_p, \|\boldsymbol{\beta}\|_q, \max_{\sigma\in\Sigma}\|\mathbf{A}_\sigma\|_q\right\},$$

where $\|\mathbf{A}_\sigma\|_q = \sup_{\|\mathbf{v}\|_q\leq 1}\|\mathbf{A}_\sigma\mathbf{v}\|_q$ is the q-induced norm.

The (p, q)-norm of a WFA A is defined as the **maximum** of three values:

1. The size of the **start vector (α)**, measured using a p-norm.
2. The size of the **end vector (β)**, measured using a q-norm.
3. The size of the **"strongest" transition matrix (A_σ)**, measured using a q-induced norm.

Essentially, the WFA is only as large as its largest component.

The Components Explained

- Vector Norms (||α||_p and ||β||_q) A vector norm is just a way of measuring its length. For example, the 2-norm is the standard ruler distance (Euclidean length), while the 1-norm is the "city block" distance. The p and q are related by the formula 1/p + 1/q = 1 and allow for different ways of measuring the start and end vectors.
- Matrix Norm (||A_σ||_q) The q-induced norm of a matrix measures its maximum "stretching power." Imagine you take all possible vectors that have a length of 1, multiply them by the matrix, and measure the lengths of the resulting vectors. The length of the longest result is the matrix's induced norm.

Example: Probabilistic Automata

This example makes the concept more concrete. A probabilistic automaton is a well-behaved WFA where weights are probabilities.

- α is a probability distribution, so its 1-norm ($||\alpha||_1$) is 1.
- β contains acceptance probabilities, so its ∞-norm (||β||_∞) is at most 1.
- The transition matrices A_σ are stochastic, meaning their ∞-norm (||A_σ||_∞) is also at most 1.

When we calculate the (1,∞)-norm, we take the maximum of these three values: max{1, ≤1, ≤1}. The result is exactly 1.

Perturbation Bounds: Automaton→Language [Bal13]

research

Suppose $A = \langle \boldsymbol{\alpha}, \boldsymbol{\beta}, \{\mathbf{A}_\sigma\} \rangle$ and $A' = \langle \boldsymbol{\alpha}', \boldsymbol{\beta}', \{\mathbf{A}'_\sigma\} \rangle$ are WFA with n states satisfying $\|A\|_{p,q} \leqslant \rho$, $\|A'\|_{p,q} \leqslant \rho$, $\max\{\|\boldsymbol{\alpha} - \boldsymbol{\alpha}'\|_p, \|\boldsymbol{\beta} - \boldsymbol{\beta}'\|_q, \max_{\sigma \in \Sigma} \|\mathbf{A}_\sigma - \mathbf{A}'_\sigma\|_q\} \leqslant \Delta$.

Claim The following holds for any $x \in \Sigma^*$:

$$|f_A(x) - f_{A'}(x)| \leqslant (|x| + 2)\rho^{|x|+1}\Delta \ .$$

Proof By induction on $|x|$ we first prove $\|\mathbf{A}_x - \mathbf{A}'_x\|_q \leqslant |x|\rho^{|x|-1}\Delta$:

$$\|\mathbf{A}_{x\sigma} - \mathbf{A}'_{x\sigma}\|_q \leqslant \|\mathbf{A}_x - \mathbf{A}'_x\|_q \|\mathbf{A}_\sigma\|_q + \|\mathbf{A}'_x\|_q \|\mathbf{A}_\sigma - \mathbf{A}'_\sigma\|_q \leqslant |x|\rho^{|x|}\Delta + \rho^{|x|}\Delta = (|x| + 1)\rho^{|x|}\Delta \ .$$

$$\begin{aligned}
|f_A(x) - f_{A'}(x)| &= |\boldsymbol{\alpha}^\top \mathbf{A}_x \boldsymbol{\beta} - \boldsymbol{\alpha}'^\top \mathbf{A}'_x \boldsymbol{\beta}'| \leqslant |\boldsymbol{\alpha}^\top (\mathbf{A}_x \boldsymbol{\beta} - \mathbf{A}'_x \boldsymbol{\beta}')| + |(\boldsymbol{\alpha} - \boldsymbol{\alpha}')^\top \mathbf{A}'_x \boldsymbol{\beta}'| \\
&\leqslant \|\boldsymbol{\alpha}\|_p \|\mathbf{A}_x \boldsymbol{\beta} - \mathbf{A}'_x \boldsymbol{\beta}'\|_q + \|\boldsymbol{\alpha} - \boldsymbol{\alpha}'\|_p \|\mathbf{A}'_x \boldsymbol{\beta}'\|_q \\
&\leqslant \|\boldsymbol{\alpha}\|_p \|\mathbf{A}_x\|_q \|\boldsymbol{\beta} - \boldsymbol{\beta}'\|_q + \|\boldsymbol{\alpha}\|_p \|\mathbf{A}_x - \mathbf{A}'_x\|_q \|\boldsymbol{\beta}'\|_q + \|\boldsymbol{\alpha} - \boldsymbol{\alpha}'\|_p \|\mathbf{A}'_x\|_q \|\boldsymbol{\beta}'\|_q \\
&\leqslant \rho^{|x|+1} \|\boldsymbol{\beta} - \boldsymbol{\beta}'\|_q + \rho^2 \|\mathbf{A}_x - \mathbf{A}'_x\|_q + \rho^{|x|+1} \|\boldsymbol{\alpha} - \boldsymbol{\alpha}'\|_p \\
&\leqslant \rho^{|x|+1}\Delta + \rho^2 \rho^{|x|-1} |x| \Delta + \rho^{|x|+1}\Delta \ .
\end{aligned}$$

We have 2 automaton, a and a’ where a’ is a wiggled distorted version of a and now we want to know the upperbound of this distortion/error.

PAC Learning for Stochastic WFA

Probabilities on strings

Stochastic Languages:

- probability distribution over all strings

- can sample finite strings and try to learn the distribution

Dynamical System:

- Probability over strings of fixed length
- Can sample(potentially infinite) prefixed and try to learn the dynamics

Hankel Estimation to find probabilities on Strings

Hankel Estimation from Strings: $\hat{f}_S(x) = \frac{1}{m}\sum_{i=1}^{m} \mathbb{I}[x^i = x]$ $\quad \hat{\mathbf{H}}(p,s) = \hat{f}_S(p \cdot s)$

Can be data inefficient as not all prefixes counted

Hankel Estimation from Prefix: $\bar{f}_S(x) = \frac{1}{m}\sum_{i=1}^{m} \mathbb{I}[x^i \in x\Sigma^*]$

Still data inefficient since we see only the finite part of strings not whole

Hankel Estimation from Substrings: $\tilde{f}_S(x) = \frac{1}{m}\sum_{i=1}^{m} |x^i|_x$ $\quad |x^i|_x = \sum_{u,v\in\Sigma^*} \mathbb{I}[x^i = uxv]$

Best efficient

Hankel Estimation from a single string for dynamical system(keep on generating): $\mathring{f}_m(x) = \frac{1}{m}\sum_{i=1}^{m} \mathbb{I}[x_i x_{i+1} \cdots \in x\Sigma^*]$

Automata Learning(Alexandra Silva)

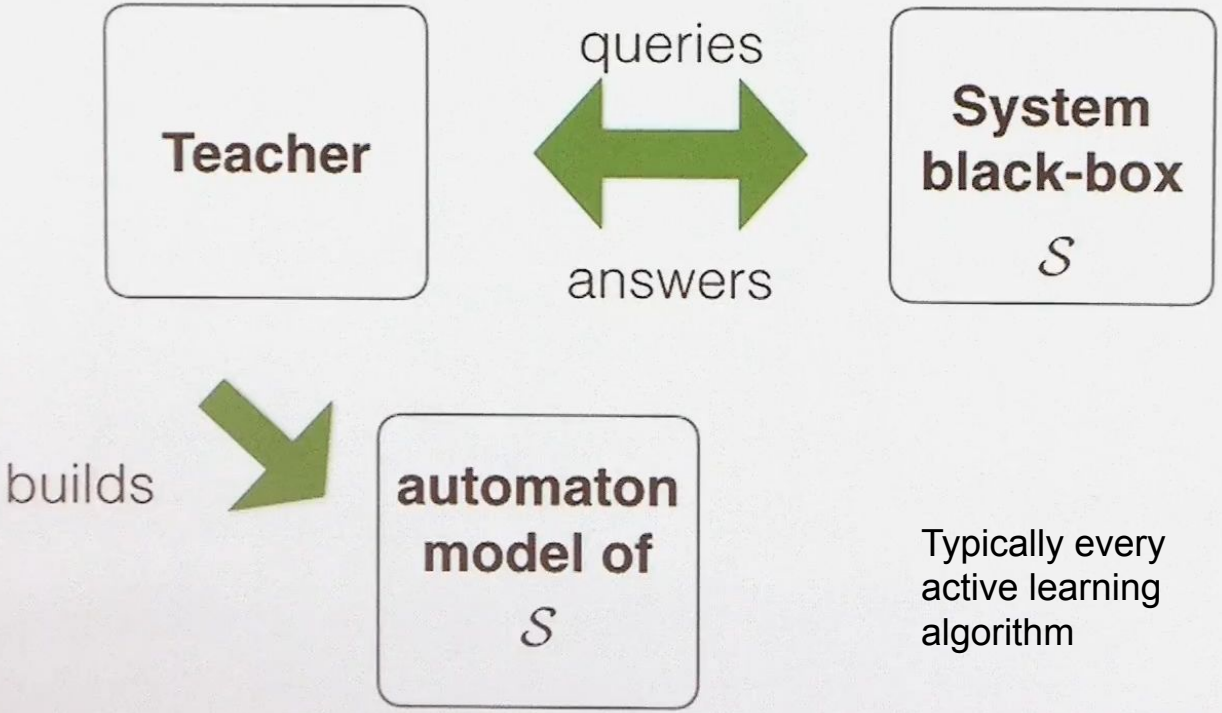
Active Learning	Passive Learning
Learning regular sets from queries and counterexamples at any time.	Record behavior of a system(log) and then infer a model that explains the system.

L* Algorithm

Well-known algorithm in **computational learning theory**, particularly in the field of **automata learning**. It's an active learning algorithm used to infer a regular language from a "teacher" who answers queries. Builds a minimal deterministic finite automaton that represents the target language(RL) by asking a minimal number of queries.

Applications- Hardware verification, network protocol
Generalisations- Mealy machines, I/O automata

L*, by example



Typically every active learning algorithm

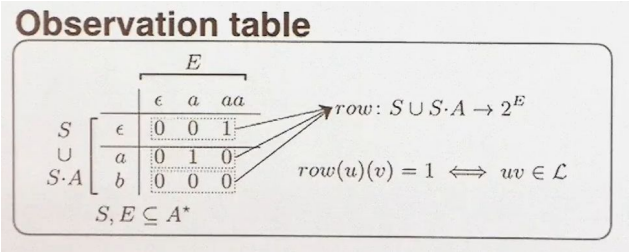
Teacher Queries

Mainly of two types-

1. **Membership Queries**: "Is this string in the language?" Is a certain word in the language.
2. **Equivalence Queries**: "Is the inferred language equivalent to the target language?" Is my hypothesis correct.

The answers will either be a YES or a NO+COUNTEREXAMPLE. Membership queries are easier to implement than equivalence as this is done by exhaustive testing(gives out a counterexample if answer is no).

The output of a system is an observation table with 0 depicting that the concatenation of row and column is present in the language.



How does L* work?

L* LEARNER

```
S, E ← {ε}
repeat
    while (S, E) is not closed or not consistent
    if (S, E) is not closed
        find s1 ∈ S, a ∈ A such that
            row(s1a) ≠ row(s), for all s ∈ S
        S ← S ∪ {s1a}
    if (S, E) is not consistent
        find s1, s2 ∈ S, a ∈ A, and e ∈ E such that
            row(s1) = row(s2) and L(s1ae) ≠ L(s2ae)
        E ← E ∪ {ae}
    Make the conjecture M(S, E)
    if the Teacher replies no, with a counter-example t
        S ← S ∪ prefixes(t)
until the Teacher replies yes to the conjecture M(S, E).
return M(S, E)
```

$$row\colon S \cup S{\cdot}A \to 2^E$$

closed

$$\forall t \in S \cdot A \quad \exists s \in S \qquad row(t) = row(s).$$

consistent

$$\forall s_1, s_2 \text{ s.t. } row(s_1) = row(s_2) \Rightarrow \forall a \in A \quad row(s_1 a) = row(s_2 a).$$

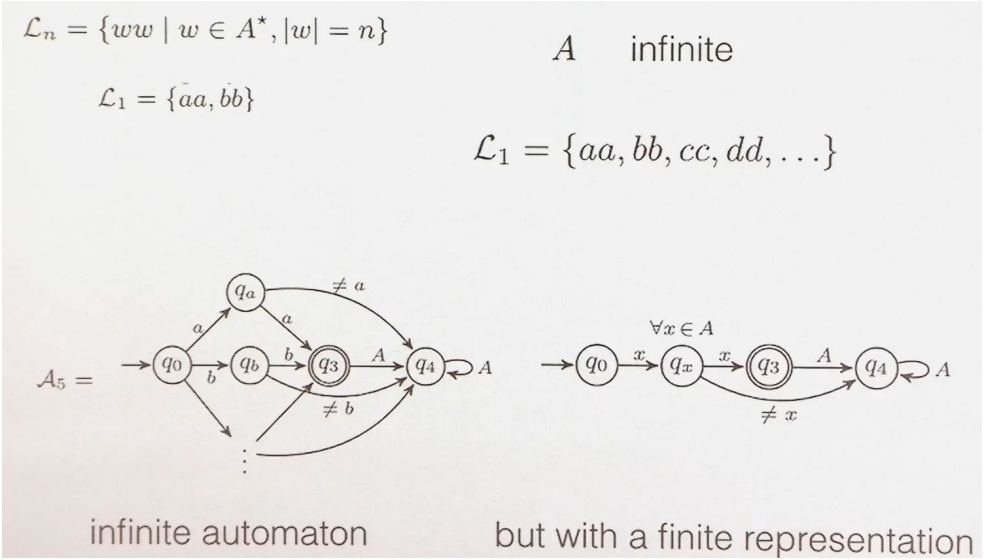
the distinct rows

- $Q = \{row(s) \mid s \in S\}$ is a finite set of states;
- $F = \{row(s) \mid s \in S, row(s)(\epsilon) = 1\} \subseteq Q$ is the set of final states;
- $q_0 = row(\epsilon)$ is the initial state;
- $\delta\colon Q \times A \to Q$ is the transition function given by $\delta(row(s), a) = row(sa)$.

Infinite Alphabets

The model can be an infinite automaton but it can still be represented finitely using certain variables or registers.

Eg -



Nominal Automata

If infinite sets have symmetries, they can be finitely represented. The core of nominal automata theory lies in the mathematical concept of **nominal sets**.

Nominal sets provide a mathematical framework for dealing with data structures that involve **names** or unique identifiers from an infinite alphabet. The key idea is that the only operation you can perform on these names is to test them for equality.

Every element x in a nominal set has a **finite support**, which is a finite set of names that are necessary to define it. If you apply a permutation of names that doesn't change any of the names in its support, the element itself remains unchanged. For example, the lambda expression `λx.x` is supported by the name `{x}`. A permutation that swaps a and b won't affect `λx.x` because neither a nor b is in its support.

Challenges in L* Algorithm

- It needs to extend over infinite sets.
- When checking for closed and consistency properties, we may need to check infinite rows.
- The response of the teacher only contains finite number of prefixes but due to infinite alphabets, an infinite S(rows) is needed.

But,

Finite Representation (P1): The key insight is that while the sets `S`, `S·A`, and `E` are infinite, they can be represented in a finite way **"up to permutations"**. This means we don't need to store every possible string; we only need to store one representative from each "orbit" of strings that are equivalent under name permutations. For example, `(n1, n2)` and `(n3, n4)` are in the same orbit, so we only need to store one of them.

Equivariance (P2): The function that defines the observation table (`row`) must be **equivariant**, meaning it respects these symmetries. If you permute the names in a string, the corresponding row in the observation table should also be a permuted version of the original row. This property ensures that the algorithm can reason about the entire infinite space by only performing checks on a finite number of representative elements.

Nominal L*

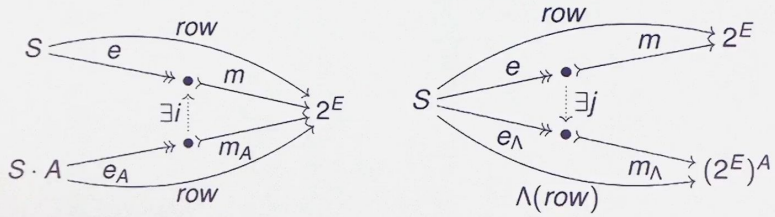
$6'$ $\quad S \leftarrow S \cup \mathtt{orb}(sa)$

$9'$ $\quad E \leftarrow E \cup \mathtt{orb}(ae)$

$12'$ $\quad S \leftarrow S \cup \mathtt{prefixes}(\mathtt{orb}(t))$

only 3 lines changed!

Categorical glasses



(S, E, row) is *closed* if for all $t \in S \cdot A$ there exists an $s \in S$ such that $row(t) = row(s)$.

(S, E, row) is *consistent* if whenever $s_1, s_2 \in S$ are such that $row(s_1) = row(s_2)$, for all $a \in A$, $row(s_1 a) = row(s_2 a)$.

Thank You